



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
FACULTAD DE INGENIERÍA DE SISTEMAS
COMPUTACIONALES



DEPARTAMENTO DE INGENIERÍA DE SOFTWARE
DESARROLLO DE SOFTWARE PARA PLATAFORMAS MÓVILES

APP GESTIÓN DE TAREAS - ROOM DATABASE CRUD

FACILITADOR: JORGE CISNEROS

GRUPO:

1SF241

INTEGRANTES:

CAMAÑO, EDWARD

8-1010-515

EXPERIENCIA:

**DESARROLLO DE UNA APLICACIÓN DE GESTIÓN DE TAREAS USANDO JETPACK
COMPOSE Y ROOM DATABASE**

FECHA DE ENTREGA:
DOMINGO 1 DE JULIO DE 2026

Asignación: Aplicación Android con Jetpack Compose y Room Database

Gestión de Tareas — TaskApp

1. Arquitectura Utilizada

La aplicación implementa el patrón MVVM (Model-View-ViewModel) siguiendo las guías oficiales de Android Architecture Components. Esta arquitectura separa claramente las responsabilidades de cada capa, facilita las pruebas unitarias y garantiza que la UI reaccione de forma reactiva a los cambios de datos.

1.1 Capas del Proyecto

Capa	Clases / Archivos	Responsabilidad
Data — Entity	Task.kt	Modelo de datos; mapea directamente a la tabla SQL
Data — DAO	TaskDao.kt	Define las operaciones SQL con anotaciones de Room
Data — Database	TaskDatabase.kt	Instancia singleton de la base de datos Room
Data — Repository	TaskRepository.kt	Abstrae el origen de datos; expone Flow a capas superiores
ViewModel	TaskViewModel.kt	Mantiene estado de la UI con StateFlow; ejecuta lógica de negocio
UI — Screens	TaskListScreen.kt	Pantalla principal; consume ViewModel con collectAsStateWithLifecycle
UI — Components	TaskItem.kt, TaskDialog.kt, EstadoVacio.kt	Composables reutilizables y sin estado propio
UI — Theme	Theme.kt	Paleta de colores Material 3 para modo claro y oscuro

1.2 Flujo de Datos

El flujo es unidireccional: Room emite actualizaciones a través de un Flow, el Repository lo expone sin transformarlo, y el ViewModel lo convierte en un StateFlow que la UI observa. Cualquier acción del usuario (insertar, eliminar, marcar completada) pasa por el ViewModel, que delega la operación al Repository en una corrutina de viewModelScope.

2. Implementación de Room Database

2.1 Entidad Task

La clase de datos Task está anotada con `@Entity` y define cinco columnas: id (clave primaria autogenerada), title, description, completed y createdAt. El campo createdAt se usa para ordenar las tareas por fecha de creación.

```
@Entity(tableName = "tasks")
data class Task(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val title: String,
    val description: String = "",
    val completed: Boolean = false,
    val createdAt: Long = System.currentTimeMillis()
)
```

2.2 DAO — Data Access Object

TaskDao define cinco operaciones sobre la tabla tasks:

Método	Anotación	Descripción
getAllTasks()	@Query	Devuelve Flow<List<Task>> ordenado: pendientes primero, más recientes al tope
getTaskById(id)	@Query	Consulta suspendida para obtener una tarea por ID
insertTask(task)	@Insert	Inserta o reemplaza si hay conflicto de ID
updateTask(task)	@Update	Actualiza todos los campos de una tarea existente
deleteTask(task)	@Delete	Elimina la fila correspondiente al objeto recibido
updateTaskCompletion(id, completed)	@Query	Actualiza solo el campo completed sin tocar los demás

2.3 Database

TaskDatabase extiende RoomDatabase y se configura como singleton con doble comprobación de bloqueo (double-checked locking) para garantizar una única instancia en toda la aplicación. La anotación `@Volatile` en la variable de instancia asegura visibilidad entre hilos.

2.4 Repository

TaskRepository recibe el DAO por constructor e implementa cada operación como función suspendida (excepto getAllTasks, que ya es un Flow). Esto aísla el ViewModel de los detalles de Room y facilitaría agregar otras fuentes de datos en el futuro (p. ej., una API REST).

3. Implementación de Jetpack Compose

3.1 Estructura de la UI

La interfaz se compone de cuatro archivos Kotlin independientes:

- TaskListScreen.kt — Pantalla raíz con Scaffold, LargeTopAppBar y LazyColumn.
- TaskItem.kt — Tarjeta de tarea con Checkbox, etiquetas de título/descripción y botones de acción.
- TaskDialog.kt — AlertDialog reutilizable para crear y editar tareas.
- EstadoVacio.kt — Pantalla de bienvenida cuando la lista está vacía.

3.2 Estado Reactivo con StateFlow

El ViewModel expone tres StateFlow que la pantalla principal consume con collectAsStateWithLifecycle (API lifecycle-aware que detiene la colección cuando la pantalla está en background):

- tareas: StateFlow<List<Task>> — lista completa desde Room.
- mostrarDialogo: StateFlow<Boolean> — controla la visibilidad del diálogo.
- tareaSeleccionada: StateFlow<Task?> — null para nueva tarea, Task para edición.

3.3 Organización Visual

La pantalla agrupa las tareas en dos secciones diferenciadas: Pendientes y Completadas. Las tareas completadas muestran el título tachado y el card cambia de color con una animación suave (animateColorAsState con 300 ms). El botón de editar se deshabilita automáticamente para tareas ya completadas.

3.4 Tema Material 3

Se definió una paleta personalizada con Indigo (#3D5AFE) como color primario y Teal (#00BFA5) como acento secundario, con soporte completo para modo claro y modo oscuro. El tema se aplica mediante TaskAppTheme en MainActivity, aprovechando isSystemInDarkTheme() para detectar la preferencia del sistema.

4. Cumplimiento de Criterios de Evaluación

Criterio	Implementación	Pts
----------	----------------	-----

Diseño de interfaz con Compose	LargeTopAppBar colapsable, ExtendedFAB, LazyColumn con animaciones, tema Material 3 claro/oscuro, estado vacío ilustrado	25
Implementación de Room Database	Entity, DAO (5 operaciones), Database singleton, Repository como capa de abstracción	25
Operaciones CRUD funcionales	Crear (dialog), Leer (LazyColumn reactivo), Actualizar (dialog de edición + toggle completado), Eliminar (con diálogo de confirmación)	30
Organización y calidad del código	MVVM estricto, nombres de variables en español, comentarios inline explicativos, sin lógica en la UI	10
Documentación y presentación	Este documento + video de demostración + repositorio en GitHub	10